GreenFog: A Framework for Sustainable Fog Computing

Adel N. Toosi[1](✉), Chayan Agarwal[1], Lena Mashayekhy[2], Sara K. Moghaddam[1], Redowan Mahmud[3], and Zahir Tari[4]

[1] Department of Software Systems and Cybersecurity, Faculty of Information Technology, Monash University, Clayton, VIC, Australia
adel.n.toosi@monash.edu

[2] Department of Computer and Information Sciences, University of Delaware, Newark, USA
mlena@udel.ed

[3] School of Electrical Engineering, Computing and Mathematical Sciences, Curtin University, Perth, Australia
mdredowan.mahmud@curtin.edu.au

[4] School of Computing Technologies, STEM College, RMIT University, Melbourne, Australia
zahir.tari@rmit.edu.a

Abstract. The alarming rate of increase in energy demand and carbon footprint of Fog environments has become a critical issue. It is, therefore, necessary to reduce the percentage of brown energy consumption in these systems and integrate renewable energy use into Fog. Renewables, however, are prone to availability fluctuations due to their variable and intermittent nature. In this paper, we propose a new Fog framework and design various optimization techniques, including linear programming optimization, linear regression estimation, and Multi-Armed Bandit (MAB) learning to optimize renewable energy use in the Fog based on a novel idea of load shaping with adaptive Quality of Service (QoS). The proposed framework, along with the optimization techniques, are tested on a real-world micro data center (Fog environment) powered by solar energy sources connected to multiple IoT devices. The results show that our proposed framework significantly reduces the difference between renewable energy generation and total energy consumption while efficiently adjusting the QoS of applications.

1 Introduction

The rise of IoT and Fog computing elicits an increase in global energy consumption and has a massive impact on the carbon footprint of the ICT (Information Communication Technology) industry. Therefore, Fog computing requires innovations in energy supply, management, and use [13]. To further lower the carbon footprint of the IoT ecosystem, it is widely accepted that renewable or green energy sources must be used as the primary power supply of Fog environments.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
J. Troya et al. (Eds.): ICSOC 2022, LNCS 13740, pp. 540–549, 2022.
https://doi.org/10.1007/978-3-031-20984-0_38

Many researchers have proposed using green energy to power Fog micro data centers [3–5,7,10,12]. They focused on various techniques to optimize renewable energy use. Li *et al.* [7] suggested task placement and scheduling. Karimiafshar *et al.* [3,5] proposed load balancing, and Toor *et al.* [10] and Karimiafshar *et al.* [4] proposed dynamic voltage and frequency scaling.

Nevertheless, the primary challenge of powering Fog environments with renewable energy sources such as solar is the *intermittency* and *variability* of power input. While different methods have been proposed to address this, a promising solution of matching demand to supply of green and renewable energy via load shaping has received little attention. In this paper, we propose a framework called *GreenFog* that conducts load shaping of a Fog environment powered by on-site renewable energy. This framework operates without the usage of a battery as we believe the installation of battery storage for Fog environments is often prohibitive [2]. Our proposed load shaping method happens through dynamic adjustment of Quality of Service (QoS) for IoT applications. According to the adjusted QoS, GreenFog horizontally scales the application software containers, the latest norm for building IoT applications. At the same time, GreenFog controls the QoS of tasks submitted to the gateway, e.g., the frame rate or image quality at which the video streaming data must be processed. Thus, the **key contributions** of the paper are as follows:

- A framework called *GreenFog* for dynamic QoS management of the IoT application based on load shaping in a renewable-powered Fog environment.
- The optimal offline model for load shaping with prior knowledge of renewables availability that sets the required QoS for the hosted IoT application.
- A fast and lightweight reactive heuristic approach based on a linear regression model and profiling to overcome the complexity of the optimal model and lack of prior knowledge of energy availability.
- A machine learning technique using Multi Armed Bandit (MAB), which automatically learns how QoS adjustment affects energy consumption and dynamically adapts QoS to maximize renewable energy utilization.
- The performance evaluation of the proposed load shaping algorithms in a practical implementation using real-world traces of renewable energy and an object detection application demonstrator in live video streams.

2 System Overview and GreenFog Framework

Our Fog environment is powered by an on-site renewable energy generation system with a grid-tied inverter that can work without batteries. The Fog environment is a multi-server cluster that monitors its own power consumption and can trace the amount of generated renewable power available for usage. It is connected to both the grid (brown energy) for reliability and an on-site renewable energy system (green energy) to reduce its carbon footprint. Note that grid energy is used whenever renewable energy is insufficient.

Figure 1 illustrates the architecture of our proposed software framework and its various software components. IoT devices are connected to a gateway device

through which they send their processing tasks (requests) to Fog. In our proposed framework, the IoT devices have APIs that allow gateway software to dynamically adapt QoS requirements remotely by setting configurations at the IoT devices. The QoS adjustment is performed within an acceptable Service Level Agreement (SLA) via a comparison between the available renewable power and the current power consumption.

We assume that the Fog environment hosts the IoT applications in the form of containerized micro-services (e.g., Docker). Fog is equipped with a cluster manager and container orchestrator software (e.g., K3s[1]) for automating deployment, scaling, and management of containers. A *master node* runs a *scheduler* program interacting with the gateway software, thereby informing the expected QoS. The scheduler also interacts with the energy metering software (*energy profile*), which monitors the available green energy and the current power consumption. Through its placement algorithm, the scheduler makes a decision regarding the *placement* of container instances (Pods in Kubernetes terminology) on different worker nodes. Another major component of GreenFog is the *scaling* component, which is responsible for the autoscaling of resources.

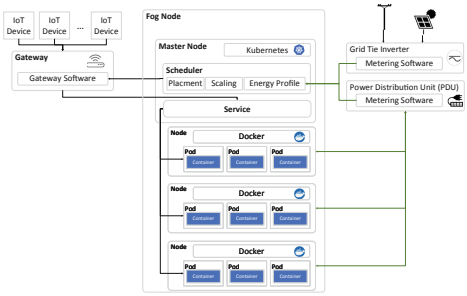


Fig. 1. GreenFog software system overview

Due to the high volume of data generated by cameras and the low latency requirements of real-time video analytics, in this paper, we mainly focus on detecting objects in live video streams [1,8]. For many video analytics applications, the data quality directly drives the QoS and resource demand. For instance, the accuracy of an object detection algorithm in a pest bird repellent system can be lowered to reduce resource usage of energy constraint devices [9]. Therefore, we set the image rate as QoS for our video analytics. In the following, we propose different optimization techniques to scale resources and adjust QoS in GreenFog.

3 Optimization Model

Assuming time is divided into time slots, we have $t = \{0, \ldots, T\}$, where T is the decision horizon. The green energy system $\mathcal{G}$ at any time t generates e^t units of electricity. We assume that the value of e^t for any time slot t is known *a priori* considering the optimal model is offline and only used as the baseline. The Fog environment $\mathcal{F}$ is a Kubernetes cluster that consists of N nodes (or worker machines). Each node $n_i \in N$ hosts at most P pods, where each pod holds a single container. Therefore, the Fog environment has the maximum of $C = P \times N$ containers in full capacity, and at each time t only some containers ($\leq C$) become

[1] K3s is a lightweight Kubernetes distribution suitable for resource-constrained environments such as Fog or Edge.

active to match the demand based on available energy e^t for that time slot. When the demand exceeds the renewable power availability, the gateway adjusts QoS to match demand with the available power. Each pod requires e_p units of electricity. Each node runs on a different virtual machine. If a node is not needed, the system simply puts the machine in a shutdown state. Each physical machine requires e_b units of energy in a shutdown state, while a running machine requires additional e_n units of energy. Each task requires e_j units of energy added to other energy terms. Moreover, there is an upper limit on the total energy consumption when running all the containers (r_{max}) and a lower limit when running one container (r_{min}). Note that in our formulations, for the sake of simplicity, we consider request rate as the only QoS factor, e.g., frame rate in our video analytics example. Other QoS factors, such as resolution, can be included in our model similarly. We also assume that each request is mapped to an IoT task in the system, e.g., an image (frame) to an image processing task.

We consider a set of homogeneous tasks generated by the IoT application. The required processing time of a task is denoted by c. The goal of GreenFog is to choose the best rate for the IoT tasks and adjust them accordingly based on the available renewable power input. We define sets of decision variables: x_i^t to determine the number of active pods at node n_i; y^t to determine the arrival rate of requests from IoT devices; and z_i^t to determine if node n_i should be on or off. Our objective is to minimize excess energy usage (brown energy usage) and determine the arrival rates of IoT tasks. We formulate this problem as an Integer Program (IP) as follows:

$$\text{Minimize} \sum_{t=1}^{T} E^t, \text{where } E^t = \sum_{n_i \in N} (e_p \cdot x_i^t + e_n \cdot z_i^t + e_b) + (e_j \cdot y^t) \tag{1}$$

Subject to:

$$E^t \geq e^t \qquad \forall t \in T \tag{2}$$

$$x_i^t \leq P \qquad \forall n_i \in N, t \in T \tag{3}$$

$$x_i^t \leq \mathbf{M} \cdot z_i^t \qquad \forall n_i \in N, t \in T \tag{4}$$

$$\sum_{n_i \in N} z_i^t \leq N \qquad \forall t \in T \tag{5}$$

$$r_{min} \leq y^t \leq r_{max} \qquad t \in T \tag{6}$$

$$c \cdot y^t = \sum_{n_i \in N} x_i^t \qquad \forall t \in T \tag{7}$$

$$x_i^t \in \mathbb{Z}_{\geq 0} \qquad \forall n_i \in N, t \in T \tag{8}$$

$$y^t \in \mathbb{Z}_{\geq 0} \qquad \forall t \in T \tag{9}$$

$$z_i^t \in \{0, 1\} \qquad \forall n_i \in N, t \in T \tag{10}$$

The objective function minimizes the total brown energy consumption subject to the following constraints. Constraint (2) guarantees that all generated renewable energy must be used. Constraint (3) guarantees that each node has at most P pods. Constraint (4) ensures that the energy cost of turning on a node should be calculated only once. This means that if any pod has already been deployed on a node, there is no additional cost for the node but for new pods. We model this

using the *Big M* method.[2] Constraint (5) ensures that the Fog environment does not exceed its capacity of N nodes. Constraint (6) ensures that the selected rate for the IoT tasks is between its minimum and maximum rates in SLA. Constraint (7) ensures that enough pods are running to service all the incoming tasks. To ensure this, we multiply the processing time for each job c with the rate of incoming tasks at time t, which is denoted by y^t, and we match this product with the total number of pods running at time t denoted by $\sum_{n_i \in N} x_i^t$. Constraints (8) and (9) guarantee that the decision variables x_i^t and y^t are non-negative integers, respectively. Constraints (10) guarantee that decision variable z_i^t is binary.

4 Linear Regression Algorithm

It is challenging to use the proposed IP model in practice. Therefore, we propose a heuristic approach based on a regression model using profiling to set QoS and scale resources. Since such a regression model only gives the rate of tasks as output, we propose a threshold-based autoscaling algorithm in Sect. 5 to scale the Kubernetes cluster. Profiling is necessary to gather the data needed to develop regression. For profiling, we identify the power consumption at different task rates. Our profiling results show that there is a simple linear correlation between the power consumption and the rate of tasks (See Sect. 7.1). Accordingly, we set α (y-intercept) and β (the slope) of the linear regression equation. We first read the current availability of green power generation. Then, we set the acceptable rate of the tasks at the gateway software in a way that Fog can fully utilize the available green energy.

5 Threshold-Based Linear Autoscaler

We propose a threshold-based autoscaler with two thresholds to keep the average pending number of tasks between the minimum and the maximum pod load thresholds. The creation/removal of a pod is performed linearly in a way that nodes should be fully packed/emptied before adding/removing a new node. If all of the currently switched-on nodes are fully utilized, and we require to add pods, we switch on a new machine (a node) and start allocating new pods to it. Using this method, we increase the number of pods allocated to a node until it cannot host any more pods; then, we add a node to the list. At any given time, there may be a node with no pods allocated to it. These nodes are removed, and we switch off machines with no active nodes to save energy. As we need to iterate through all P pods running on maximum N nodes, the time complexity of the Autoscaler algorithm is $O(NP)$.

6 Multi-Armed Bandit Approach

Specific knowledge, preliminary to the implementation, is required by the IP and linear regression algorithms. To address this problem, we propose an online

[2] Big M method is a method of solving linear programming problems using simplex.

machine learning model based on the Multi-Armed Bandit (MAB) approach [6] using the Upper Confidence Bound (UCB) method. In our MAB problem, the agent needs to choose among five different actions (arms) referring to varying the rate of tasks (QoS): 1) increase rate high, 2) increase rate low, 3) do nothing, 4) decrease rate low, and 5) decrease rate high. All these actions have rewards set between 0 to 1, initially valued at 0.5. The obtained rewards depend on the difference between the available renewable energy and the energy consumed. When the system is stable, our MAB-based algorithm computes the reward for the last choice made using a reward function. The stable condition is achieved when there is no change in the number of pods by the Autoscaler for the last four consecutive time slots. MAB algorithms generally have polynomial time complexity.

7 Performance Evaluation

Hardware: The setup contains 3 Dell servers managed by OpenStack and connected to Eaton ePDUs, allowing us to monitor the power consumption of the servers. A total of 5 Virtual Machines (VMs) can be hosted, out of which one acts as a Kubernetes master and others as Kubernetes nodes (workers). The master VM runs on the main server alone, and the rest are running on the other two physical servers (2 VMs on each physical server). Each VM has 16 GB of Memory, 8 VCPUs and 160 GB of Disk space. We limit the maximum resources (CPU, Memory, etc.) that can be used by each pod. Every VM can accommodate up to 5 pods, including the master VM, with a total of 25 pods for all VMs. Apart from the servers, we use a group of three Raspberry Pi Model 3B+ to act as IoT devices. The Raspberry Pis use the Pi Camera Module v2.

Software: The Raspberry Pis run a client code taking images and sending them to the Kubernetes service running on the master node. They also run a flask server with RESTful APIs through which we can set the image rate. The flask server accepts the interval in seconds that images are sent (minimum of 0.3 s). The maximum power consumption of our system is around 1 KW, whereas the maximum renewable energy available in the traces is also roughly 1 kWh as anything above will not be utilized by the servers. We have not taken into account the energy needed for cooling or other power consumption sources and renewable power is solely utilized for server power consumption. We deploy YOLO V3 (You Only Look Once), an object detection technology, as a service in the pods to process the image data sent by the Raspberry Pis to the endpoint of the Kubernetes service. We implemented the Gateway software which interacts with the master node and the Raspberry Pis, that evenly sets the rate provided by the master node across all three Raspberry Pis.

Renewable Energy Trace: The renewable energy traces used for all the experiments are based on the availability of solar energy for a location of a data center in Lyon, France. We used the data traces for the Global Horizontal Irradiance (GHI) at the location to calculate the output for the solar photovoltaics (PV) power with one-hour granularity between the 20th and 21st of September, 2007. For more details see our previous paper [11].

7.1 Results

Linear Regression with Linear Autoscaling: The linear regression model requires profiling for different rates of images while the Autoscaler changes the number of pods according to the average number of pending tasks. The lower and upper thresholds for the Autoscaler are set at 10 and 15, respectively. Then, we identify the power consumption for different rates. For each rate, we wait until the Fog environment becomes stable. We fit a linear regression model where the power consummation is the explanatory variable and the rate (Images per Second) is the dependent variable. Figure 2 shows how well the regression model predicts the power consumption compared to the actual data (α (y-intercept) $= -0.2535$ and β (slope) $= 0.00324$).

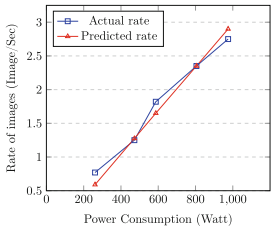
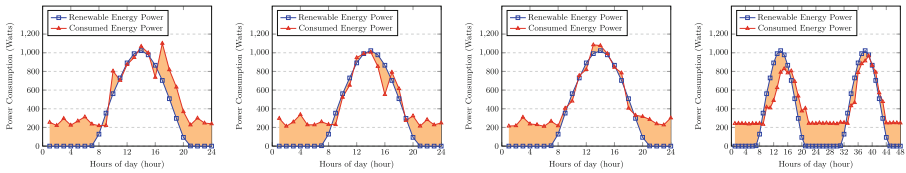


Fig. 2. Regression model predictions vs actual values

We ran the linear regression model with the Autoscaler. To reduce the duration of experiments, we scale every hour in renewable energy traces into a 15-min interval. However, we report the results based on the 24-h scale. The total absolute energy difference between the available renewable energy and the energy consumed in the 24-h test is 4.886 kWh as shown by the shaded area in Fig. 3(a). The sudden spikes and dips deviating from the renewable energy are due to Autoscaler's performance. As it increases the number of pods linearly (1 after another), getting the correct number of pods to handle the incoming tasks might take some time. Therefore, the average pending tasks in each pod could fluctuate to extreme limits. In addition, a node shutdown might take 10–12 min, adding a considerable latency to reach stability.

IP Model with Linear Autoscaling: We use the IP model from Sect. 3 with the default Linear Autoscaler from Sect. 5. The IP model requires certain information about the system, such as energy consumed per pod, energy consumed by the servers when running or shut down, the energy needed to service a task, and the processing time it takes for a task. To obtain the energy consumed per pod, we ran the system with one VM (master) with two pods running, and then we scaled the deployment by adding another pod to the same VM



(a) (b) (c) (d)

Fig. 3. GreenFog with different optimization methods: (a) Linear regression with linear autoscaler, (b) IP with linear autoscaler, (c) IP with direct scaler, and (d) MAB with linear autoscaler

to record the rise in energy consumption. For energy consumed by the servers, we first switched off the server and recorded the power consumption; then, we switched on the system and noted the difference in power consumption. To figure out the energy needed to service a task, we ran GreenFog with Linear Autoscaler and then we sent one image every second and waited until the cluster becomes stable and noted down the power consumption; from this value we subtracted the values obtained previously like the energy per pod and the energy per node in On/Off state appropriately to get the energy per task.

If such information is known, this model can provide the number of servers that should be running, how many pods in each server are required, and also the acceptable rate of jobs which helps in setting QoS. In this experiment, we ignore scaling information provided by the IP model and we use the Linear Autoscaler instead. The results are shown in Fig. 3(b), illustrating that the IP model with Linear Autoscaler has 16% less absolute energy consumption difference compared to the Linear Regression model for the same data by being 4.095 kWh compared to the 4.886 kWh, respectively. In the next section, we investigate the performance of the IP model if scaling information is provided by the model itself.

IP with Direct Scaler: Figure 3(c) shows the results of the experiments for the IP model with its scaling information called *Direct Scaler*, which utilizes all the decision variables from the IP model. The Direct Scaler receives the information owing to which nodes are to be switched On/Off, and the number of pods that should be running in each node from the IP model. The IP model also sets the rate of images (QoS) that should be sent to the gateway software. Figure 3(c) shows a considerably smooth power consumption pattern with a lower number of spikes compared to the previous methods. The IP model with Direct Scaler has 25% less absolute energy consumption difference compared to the Linear Regression model for the same data by being 3.677 kWh compared to the 4.886 kWh, respectively. Similarly, it is 10% less compared to 4.095 kWh for the IP model with Linear Autoscaler.

MAB Model with Linear Autoscaling: Figure 3(d) depicts power consumption for the GreenFog Framework, where the MAB model with the Linear Autoscaler is used. We conducted two days of experiments, where the first day would provide the MAP model with the opportunity to learn the system's patterns. The renewable energy data is the same across two days to show the best achievable results by MAB. The total absolute energy difference in the second day is 4.164 kWh. This is 14% better than the linear regression model and about the same as the IP model with Linear Autoscaler. However, as we expected, the baseline IP model with Direct Scaler outperforms the MAB model by 11%. Given proper training data, the MAB model outperforms other approaches.

8 Conclusions and Future Work

We proposed GreenFog, a framework to optimize green energy use for Fog environments with on-site renewable electricity generation. We designed and implemented an Integer Programming optimization model with exact energy requirements knowledge for different components to use as a baseline. We proposed a heuristic approach based on linear regression to overcome the limitations of the IP. Since the performance of the Linear Regression model relies on accurate profiling, we proposed a machine learning model based on the Multi-Armed Bandit problem. We evaluated and validated the GreenFog framework equipped with these optimization techniques on a real testbed. We measured the actual power usage and response times with a practical IoT application and realistic traces of renewable energy. In the future, we are interested in developing a model to work based on forecasting future renewable energy availability. We further look into solving this problem with other machine learning methods, such as deep reinforcement learning.

References

1. Ananthanarayanan, G., et al.: Real-time video analytics: the killer app for edge computing. Computer **50**(10), 58–67 (2017)
2. Goiri, I.N., et al.: GreenSlot: scheduling energy consumption in green datacenters. In: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2011. Association for Computing Machinery, New York (2011). https://doi.org/10.1145/2063384.2063411
3. Karimiafshar, A., Hashemi, M.R., Heidarpour, M.R., Toosi, A.N.: An energy-conservative dispatcher for fog-enabled IIoT systems: when stability and timeliness matter. IEEE Trans. Serv. Comput., 1 (2021). https://doi.org/10.1109/TSC.2021.3114964
4. Karimiafshar, A., Hashemi, M.R., Heidarpour, M.R., Toosi, A.N.: Effective utilization of renewable energy sources in fog computing environment via frequency and modulation level scaling. IEEE Internet Things J. **7**(11), 10912–10921 (2020)
5. Karimiafshar, A., Hashemi, M.R., Heidarpour, M.R., Toosi, A.N.: A request dispatching method for efficient use of renewable energy in fog computing environments. Futur. Gener. Comput. Syst. **114**, 631–646 (2021)
6. Lattimore, T., Csaba, S.: Bandit Algorithms. Cambridge University Press, Cambridge (2020)
7. Li, W., et al.: On enabling sustainable edge computing with renewable energy resources. IEEE Commun. Mag. **56**(5), 94–101 (2018)
8. Ma, W., Mashayekhy, L.: Quality-aware video offloading in mobile edge computing: a data-driven two-stage stochastic optimization. In: Proceedings of the 14th IEEE International Conference on Cloud Computing, pp. 594–599 (2021)
9. Mahmud, R., Toosi, A.N.: Con-Pi: a distributed container-based edge and fog computing framework. IEEE Internet Things J. **9**(6), 4125–4138 (2022)
10. Toor, A., et al.: Energy and performance aware fog computing: a case of DVFS and green renewable energy. Future Gener. Comput. Syst. **101**, 1112–1121 (2019)
11. Toosi, A.N., Qu, C., de Assunção, M.D., Buyya, R.: Renewable-aware geographical load balancing of web applications for sustainable data centers. J. Netw. Comput. Appl. **83**, 155–168 (2017)

12. Zeng, D., Gu, L., Yao, H.: Towards energy efficient service composition in green energy powered cyber-physical fog systems. Futur. Gener. Comput. Syst. **105**, 757–765 (2020)
13. Zhang, G., Chen, Y., Shen, Z., Wang, L.: Distributed energy management for multiuser mobile-edge computing systems with energy harvesting devices and QoS constraints. IEEE Internet Things J. **6**(3), 4035–4048 (2019)